Planejamento: CandyMatch (Match-3 em Java + libGDX)

Trabalho de Estrutura de Dados 1 (ED1) · Jogo de combinação inspirado em Candy Crush · Dupla: *[nome A]* e *[nome B]* · Prazo: *[preencher]*

1. Visão geral do jogo

Mecânica base: tabuleiro 8x8 com gemas coloridas. O jogador troca duas gemas vizinhas. Se a troca formar 3 ou mais gemas iguais em linha ou coluna, elas somem, as de cima caem e novas gemas entram. Combos em cascata dão mais pontos.

Fluxo de uma jogada

```
Clique/arraste → Valida troca → Procura matches → Remove gemas
        ↑                                              ↓
   Fim de jogo? ← Atualiza pontos ← Refill ← Gravidade
        (se houve match, repete o ciclo até estabilizar)
```

Fluxo de telas

```
Menu Principal ──→ Seleção de Nível ──→ Tela do Jogo ──→ Game Over / Vitória
      |                                                        |
      └──────────────→ Ranking ←──────────────────────────────┘
```

2. Estrutura de pastas do projeto

```
CandyMatch/
├── core/src/main/java/com/seujogo/
|   ├── MainGame.java              → classe principal (Game do libGDX)
|   ├── screens/                   → telas do jogo
|   |   ├── MenuScreen
|   |   ├── LevelSelectScreen
|   |   ├── GameScreen
|   |   ├── GameOverScreen
|   |   └── RankingScreen
|   ├── model/                     → dados do jogo (sem lógica pesada)
|   |   ├── Gem, GemType (enum)
|   |   ├── Board                  → contém a MATRIZ
|   |   ├── Position, Match
|   |   ├── Level
|   |   └── ScoreEntry
|   ├── logic/                     → regras do jogo
```

```
|   |   ├── MatchFinder
|   |   ├── GravityHandler
|   |   ├── BoardRefiller
|   |   ├── HintFinder
|   |   ├── ScoreCalculator
|   |   └── UndoManager
|   ├── structures/               → estruturas de dados próprias (genéricas)
|   |   ├── Node<T>
|   |   ├── MyLinkedList<T>
|   |   ├── MyStack<T>
|   |   └── MyQueue<T>
|   ├── algorithms/               → busca e ordenação genéricas
|   |   ├── Sorter
|   |   └── Searcher
|   ├── io/                       → arquivos
|   |   ├── RankingFileManager
|   |   ├── SaveGameManager
|   |   └── LevelLoader
|   └── ui/                       → parte visual
|       ├── BoardRenderer
|       ├── InputHandler
|       ├── AnimationManager
|       └── Hud (pontos, jogadas, botões)
├── assets/
|   ├── images/   (gemas, fundo, botões)
|   ├── sounds/   (efeitos, música)
|   ├── fonts/
|   └── levels/   (level1.txt, level2.txt...)
└── lwjgl3/                       → launcher desktop (gerado pelo libGDX)
```

3. Checklist do professor: o que implementar e onde

Requisito	O que implementar	Onde
Qualidade do jogo	Jogo jogável do começo ao fim, com menu, níveis, pontuação, fim de jogo, animações e sons.	`screens/*`, `ui/*`
Vetor ou Matriz	Matriz `Gem[8][8]` representando o tabuleiro. Vetores para os tipos de gema, configuração dos níveis e top 10 do ranking.	`model/Board`, `model/Level`

Requisito	O que implementar	Onde
Generic	Todas as estruturas com `<T>` (Node, lista, pilha, fila). Ordenação com `<T extends Comparable<T>>` ou `Comparator<T>`.	`structures/*`, `algorithms/*`
Arquivos	Salvar/carregar ranking (`.txt` / `.csv`), salvar/continuar partida (estado do tabuleiro) e ler níveis de arquivos de configuração.	`io/*`
Pilha	Botão **Desfazer jogada**: empilha cada estado/jogada e desempilha ao desfazer.	`logic/UndoManager` + `structures/MyStack`
Fila	Fila de eventos/animações: matches encontrados e quedas são enfileirados e processados em ordem (essencial para cascatas).	`ui/AnimationManager` ou `logic/` + `structures/MyQueue`
Lista encadeada	Lista de matches encontrados em cada rodada e lista do ranking carregado do arquivo.	`structures/MyLinkedList`, usada em `MatchFinder` e `RankingFileManager`
Algoritmo de busca	**Busca binária** no ranking (ordenado por nome) para consultar o recorde de um jogador. **Busca sequencial** na `HintFinder` para achar uma jogada válida e dar dica.	`algorithms/Searcher`, `logic/HintFinder`
Algoritmo de ordenação	**MergeSort ou QuickSort** para ordenar o ranking por pontuação (e por nome, para habilitar a busca binária).	`algorithms/Sorter`, `RankingScreen`

> **Importante:** confirmar com o professor se é permitido usar `java.util` (ArrayList, Stack, etc.). Na maioria das disciplinas de ED1 as estruturas precisam ser implementadas à mão, e este plano já considera isso.

4. Qualidade do jogo: priorização

MVP (obrigatório para funcionar)

- Tabuleiro com troca de gemas por clique/arraste
- Detecção de matches, gravidade e refill com cascata
- Pontuação e limite de jogadas
- Menu, tela de game over e ranking

Nível 2 (dá cara de jogo de verdade)

- Animações de troca, queda e explosão
- Sons e música
- Vários níveis com metas diferentes (ex.: "faça 1000 pontos em 15 jogadas")
- Botões de desfazer e dica

Nível 3 (diferencial, se sobrar tempo)

- Gemas especiais (match de 4 vira bomba de linha, match de 5 vira bomba de cor)
- Partículas e efeitos visuais
- Salvar e continuar partida

5. Divisão de trabalho (2 pessoas)

Fase	Pessoa A: Lógica e Estruturas	Pessoa B: Interface e Arquivos
Fase 1	Estruturas genéricas (lista, pilha, fila)	Setup do libGDX, telas vazias, navegação
Fase 2	`Board`, `MatchFinder`, gravidade, refill	`BoardRenderer`, `InputHandler` (desenhar e clicar)
Fase 3	`UndoManager`, `HintFinder`, pontuação	Animações, HUD, sons
Fase 4	`Sorter` e `Searcher`	`RankingFileManager`, `LevelLoader`, `RankingScreen`
Fase 5	Integração, testes e bugs (juntos)	Integração, polimento e apresentação (juntos)

Contrato inicial: combinar logo no começo o que o `Board` expõe (trocar gemas, perguntar se há match, aplicar gravidade). Assim a Pessoa B desenha na tela sem esperar a lógica ficar pronta.

6. Etapas do projeto

1. **Fase 0: Setup.** Criar o projeto com o **gdx-liftoff** (gerador oficial do libGDX), repositório no GitHub e definir o padrão de nomes (português ou inglês).
2. **Fase 1: Estruturas.** Implementar e testar as estruturas genéricas isoladamente (no console).
3. **Fase 2: Lógica no console.** Fazer o jogo funcionar sem gráficos (imprimir a matriz no terminal). Isso evita misturar bug de lógica com bug visual.
4. **Fase 3: Interface libGDX.** Desenhar o tabuleiro, capturar cliques e ligar na lógica.
5. **Fase 4: Pilha, fila e arquivos.** Desfazer, fila de eventos, salvar/carregar.
6. **Fase 5: Ranking, busca e ordenação.** Fechar os requisitos do professor.
7. **Fase 6: Polimento.** Animações, sons, níveis, bugs e ensaio da apresentação.

7. Cronograma

Fase	Início	Fim	Responsável	Status
0. Setup			Ambos	
1. Estruturas				
2. Lógica no console				
3. Interface libGDX				
4. Pilha, fila e arquivos				
5. Ranking, busca e ordenação				
6. Polimento e apresentação			Ambos	

8. Pontos de atenção

- No libGDX, `Gdx.files.internal` é somente leitura. Para **escrever** o ranking e o save, use `Gdx.files.local`.
- Manter lógica (`model`, `logic`, `structures`, `algorithms`) sem dependência do libGDX, para poder testar no console.
- Confirmar com o professor as regras sobre `java.util` e sobre o formato da apresentação.
- Fazer commits pequenos e frequentes, com cada pessoa trabalhando em pastas diferentes para evitar conflitos.